

Chapter 4 · Lesson 1 — The useState Mental Model

Chapter 4 · Lesson 1 — The useState Mental Model

Goal: completely demystify useState. By the end of this lesson you can answer “what does useState do?” in a way that gets you hired, and you can predict exactly what happens to your component when state changes.

What is “state”?

In a React app, every piece of data falls into one of two categories:

- **Props** — data passed in from a parent component. The component cannot change props directly. They flow downward.
- **State** — data the component owns and can change. When state changes, the component re-renders.

Then there’s a third bucket that beginners sometimes confuse:

- **Derived data** — data computed from props or state. Not state itself; just a function of state. (Example: `teamsWithScores` in Chapter 3 — computed from `TEAMS`, not stored separately.)

Senior rule: if a piece of data can be **derived** from existing state or props, **don’t put it in state**. Compute it on every render. Storing it duplicates the source of truth and creates synchronization bugs. State is for the *minimum* set of changeable values.

The interview question: “How do you decide what should be state?” Answer: “I ask ‘is this data the source of truth, or can it be computed from something else?’ Only sources of truth become state.”

What is useState?

useState is a **React hook** — a function with a special property that React tracks per-component-instance.

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

Six lines, complete React component with state.

Anatomy of useState

```
const [count, setCount] = useState(0);
```

Three things in this line:

1. **useState(0)** — call the hook with an *initial value* (0). React allocates a “state cell” for this component instance.
2. **useState returns an array of two items** — [currentValue, setterFunction].
3. **Array destructuring** — `const [count, setCount]` pulls the two items out and names them.

By convention, the setter is named `set` + capitalized state name. `setCount` for `count`. `setUser` for `user`. Stick with this convention; everyone reading your code expects it.

What does the setter do?

When you call `setCount(5)`, two things happen:

1. React **updates the state cell** to 5.
2. React **schedules a re-render** of this component.

That's it. It doesn't immediately mutate `count` (the variable). It doesn't return the new value. It schedules React to re-execute your component function with the new state.

The classic gotcha:

```
function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
    console.log(count);    // logs the OLD value, not the new one!
  }

  return <button onClick={handleClick}>{count}</button>;
}
```

Why? Because `count` is a `const` captured from the current render. The setter changes the underlying state cell, but the variable in this closure still holds the old value until the **next** render produces a new closure with the new value.

The mental model: **state is not a variable you mutate; it's a value you read this render and request a new value for next render.**

The re-render cycle

When state changes, React **re-renders the component** — it calls the function again from the top.

```
function Counter() {
  console.log('rendering Counter');
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}
```

Initial render: `'rendering Counter'` logs once. Browser shows `"0"`.

User clicks: `setCount(0 + 1)` is called. React schedules a re-render.

Re-render: `'rendering Counter'` logs again. `useState(0)` runs **again** but returns the new state value (1, not 0). The button now shows `"1"`.

User clicks again: `setCount(1 + 1)` schedules re-render. Function re-runs. `useState` returns 2. Button shows "2".

Every state change triggers a re-render. Every render runs the function from the top. Variables inside the function are fresh every time. The persistent state is the value React holds for you.

The mental loop

1. React calls your function (render).
2. Your function returns JSX.
3. React updates the DOM to match the JSX.
4. User does something (clicks, types).
5. Event handler fires; calls `setSomething(...)`.
6. React queues a re-render.
7. Go to 1.

That's the whole React lifecycle in seven steps. Memorize it. **Every UI bug you ever encounter has its root in some step of this loop.** Knowing the loop = knowing where to look.

Hooks: the rules

There are exactly **two rules of hooks**, and they exist because of how React tracks state cells.

Rule 1: Call hooks at the top level. Never inside loops, conditions, or nested functions.

```
// BROKEN
function Foo({ show }) {
  if (show) {
    const [x, setX] = useState(0);    // ❌ conditional hook
  }
}

// CORRECT
function Foo({ show }) {
  const [x, setX] = useState(0);      // ✅ always called
```

```

    if (show) {
      return <div>{x}</div>;
    }
    return null;
  }
}

```

Why? React tracks hooks **by call order**. Render 1: `useState` is the first hook. Render 2: `useState` is the first hook. They match — same state cell. If on render 3 the condition is false and `useState` isn't called, React loses track of which state cell maps to which `useState` call. Bugs ensue.

Rule 2: Call hooks only from React function components or other hooks.

You can call `useState` inside a function component (`function Foo()`) or inside a custom hook (`function useFoo()`). Not inside a regular utility function. The reason is the same — React's per-component tracker only runs during component rendering.

These rules are enforced by the `eslint-plugin-react-hooks` package, which is installed by default in `create-next-app`. If you violate them, ESLint yells at you. **Listen to ESLint.**

Hooks are detectable by name

The convention: hooks start with `use`. `useState`, `useEffect`, `useMemo`, `useReducer`, `useRef`, `useContext`, etc. **Custom hooks must also start with `use`** — that's how the linter knows to enforce the rules on them.

Initial value can be expensive — use a function

```

// Expensive initial value computed every render (wasteful)
const [data, setData] = useState(JSON.parse(localStorage.getItem('data')) ||

// Lazy initialization — function is called only on first render
const [data, setData] = useState(() => JSON.parse(localStorage.getItem('data')

```

The first form re-evaluates `JSON.parse(...)` on every render (though `useState` ignores the result on subsequent renders). The second form is **lazy** — React only calls the

initializer once.

For cheap initial values (`useState(0)`, `useState('')`), don't bother. For anything that involves computation, IO, or `JSON.parse`, use the function form.

Multiple state values

You can call `useState` as many times as you want in one component:

```
function Form() {  
  const [name, setName] = useState('');  
  const [email, setEmail] = useState('');  
  const [age, setAge] = useState(18);  
  // ...  
}
```

That's totally fine. Each call gets its own state cell. **Don't try to combine them into one giant `useState({ name, email, age })` unless they truly belong together** — the multi-call form is more readable and lets each piece update independently.

When *should* you combine into one `useState`? When the values are tightly coupled and always update together. Example: `useState({ width: 0, height: 0 })` for a measured DOM element. They're always set at the same time, so one state cell is fine.

State updates are batched

If you call multiple setters in the same event handler, React batches them into one re-render:

```
function handleClick() {  
  setCount(count + 1);  
  setName('clicked');  
  setMessage('hello');  
  // Only ONE re-render happens, after all three setters resolve.  
}
```

This is **automatic batching**, on by default in React 18+. It's a performance feature you barely notice — but if you're debugging strange behavior, it's good to know.

State updates can be functional

```
setCount(count + 1);           // value form – uses the count from this closure
setCount(prev => prev + 1);    // functional form – receives the latest state
```

When does it matter? When you're updating state based on previous state **AND** the update might happen multiple times before the re-render:

```
// PROBLEM: stale closure
function handleClick() {
  setCount(count + 1);
  setCount(count + 1);    // count is still the OLD value here, so this sets
  setCount(count + 1);    // same – final result is old+1, not old+3
}

// FIX: functional updates
function handleClick() {
  setCount(prev => prev + 1); // prev is the latest queued value
  setCount(prev => prev + 1);
  setCount(prev => prev + 1); // final result is old+3
}
```

Rule of thumb: **if the new state depends on the old state, use the functional form.** It's safer in async / batching scenarios.

What state is in <SnookerFantasyLeague>?

Now apply this thinking to our app. From Chapter 1, the user stories are:

- See standings
- See matches
- See predictions
- See player drill-downs
- See analytics

Read those carefully. **What in our app changes over time as the user clicks?**

Two things:

1. **Which tab is active.** The user clicks “Predictions” → `activeTab` becomes `'predictions'`. That’s state.
2. **Which team is selected (for predictions/players drill-down).** The user clicks “Invincibles” in the standings → `selectedTeam` becomes the `Invincibles` object. That’s state.

Anything else? No. The match data doesn’t change (it’s typed into a file). The team picks don’t change. The scores are derived from the data. Nothing else changes from user interaction.

Two pieces of state. That’s our entire global state surface. Compare to a Redux app where you might have 50 slices — for *this* app, two `useState` calls suffice.

This is the senior answer to “how do you manage state?” — start by enumerating what *changes*. The answer is usually a smaller list than juniors expect.

A small example: the round filter

Some components have **local** state that doesn’t need to live at the top. Take the `<MatchesTab>` (which we’ll build properly in Chapter 5) — it has a tab strip for R1 / R2 / QF / SF / Final. Which round is showing?

That state could live in `<SnookerFantasyLeague>`, but **nothing else in the app cares which round `MatchesTab` is showing**. It’s a UI detail of `MatchesTab` itself. So it lives there:

```
'use client';
import { useState } from 'react';

export default function MatchesTab() {
  const [round, setRound] = useState<'r1' | 'r2' | 'qf' | 'sf' | 'f'>('r1');
  // ...
}
```

The TypeScript type `'r1' | 'r2' | 'qf' | 'sf' | 'f'` is a **string union** — only those five values are allowed. Pass `'foo'` and TS yells. **Use string unions instead of plain string whenever your state has a fixed set of values.** It catches typos, helps autocomplete, makes refactoring safe.

This is the **lowest common ancestor** rule from Chapter 1: state lives at the lowest compo-

ment that needs to read or write it. For tab-internal state, that's the tab itself. For app-wide state, it's the orchestrator.

When state belongs in a parent (lifting state up)

The other situation: **two siblings need to share state**. Example: clicking a row in `<StandingsTab>` should switch the active tab to `<PredictionsTab>` and pre-select the team.

Three components need to know:

- `<SnookerFantasyLeague>` — owns `activeTab` to decide which tab to render.
- `<StandingsTab>` — needs to *trigger* the change.
- `<PredictionsTab>` — needs to *read* `selectedTeam`.

The rule: **state lives at the lowest common ancestor of every component that needs it**. Lowest common ancestor of those three is `<SnookerFantasyLeague>`. So state lives there. `<StandingsTab>` gets a callback prop (`onTeamClick`) that calls the parent's setter. `<PredictionsTab>` gets the value as a prop (`selectedTeam`).

Lesson 2 walks through this in detail. For now, internalize the principle: **state location is determined by the data, not by which component "owns" the feature**.

The hook isn't magic — it's a closure trick

You don't *need* to know how `useState` works internally to use it. But the explanation closes the mental gap for many beginners:

```
// Pseudocode for what React does internally
let stateCells: any[] = [];
let cellIndex = 0;

function useState<T>(initial: T): [T, (newVal: T) => void] {
  const i = cellIndex;
  if (stateCells[i] === undefined) stateCells[i] = initial;
  const setter = (newVal: T) => {
    stateCells[i] = newVal;
    scheduleRerender();
  };
  cellIndex++;
}
```

```

    return [stateCells[i], setter];
  }

  function render(component) {
    cellIndex = 0;           // reset before each render
    return component();
  }

```

That’s a simplified version — React actually uses linked lists of “fiber” hooks per component instance — but the spirit is right. **State cells indexed by call order, reset on each render, looked up by position.** That’s why hooks must be called in the same order every render. Now you know.

Vibe prompt you would have used

*“I’m going to add interactivity to my app for the first time. Here’s the data flow: clicking a row in <StandingsTab> should set a `selectedTeam` and switch the active tab to <PredictionsTab>. The state for `activeTab` and `selectedTeam` should live in the parent (<SnookerFantasyLeague>). Add `useState` for both, pass them down where needed, and pass setters as callbacks to <StandingsTab>. **Don’t use Context, don’t use Redux** — just use `useState` in the parent and prop drilling. Mark the parent file with `‘use client’`.”*

The “Don’t use Context, don’t use Redux” line is critical. LLMs love to over-engineer. Telling them what *not* to use is half the prompt.

CHECK YOURSELF

1. `const [count, setCount] = useState(0);` — what does each side of the destructuring do? What’s the type of `count`? What’s the type of `setCount`?
2. You write `setCount(count + 1); console.log(count);` — what does the console log? Why?
3. The two rules of hooks are: **top-level only, and React-functions-or-hooks only.** Why does the first rule exist? (Hint: it’s about React’s tracking mechanism.)
4. You’re building a search input that filters a list. Should `searchQuery` be state? Should the *filtered list* be state? Why or why not?

5. In our snooker app, should `teamsWithScores` (the sorted, decorated team list) be state? Why or why not?

When you've answered these, head to [02-the-orchestrator-pattern.md](#) — where we lift state up and finally make the app feel like an app.